Data and Artificial Intelligence

Cyber Shujaa Program

Week 7 Assignment
Linear Regression Models

Student Name: Amidu Dabor
Student ID: CS-DA01-25031
Date: 2 July, 2025

TABLE OF CONTENTS

1.0 INTRODUCTION

This project focused on implementing Linear Regression models to predict house prices using Python and Scikit-learn. The task involved both Simple Linear Regression (SLR) using a single feature (area) and Multiple Linear Regression (MLR) using multiple features (area, bedrooms, and age). The goal was to explore real-world data, prepare it for modeling, evaluate regression performance, and visualize predictions in a way that could be shared and presented professionally.

Objectives

The key objectives of this project were to:

1. Explore and analyze a real-world dataset related to house prices.
2. Prepare and clean the dataset, including handling missing values.
3. Build a Simple Linear Regression model to predict price based on area.
4. Build a Multiple Linear Regression model to predict price using area, bedrooms, and age.
5. Evaluate both models using key performance metrics: MAE, MSE, RMSE, and R^2 score.
6. Visualize regression results, including scatter plots and regression lines.

2.0 TASK COMPLETION

2.1 Dataset Overview

Two datasets were used:

- *homeprices.csv* for Simple Linear Regression:
  Variables: area, price
- *homeprices-m.csv* for Multiple Linear Regression:
  Variables: area, bedrooms, age, price

Both datasets contained 1,000 observations. Initial exploration included reviewing the first few rows, summarizing key statistics, and checking for missing values.

```python
self.df = pd.read_csv(self.data_path)
print("First 5 Rows:\n", self.df.head())
print("\nSummary Statistics:\n", self.df.describe())
print("\nMissing Values:\n", self.df.isnull().sum())
```

Simple Linear Regression

```python
model = LinearRegressionModel("homeprices.csv", x_columns="area", y_column="price")
model.load_and_explore_data()
model.prepare_data()
model.train_model()
model.evaluate_model()
model.visualize_results()
```

[2] ✓ 0.2s Python

```
First 5 Rows:
    area   price
0  2600  550000
1  3000  565000
2  3200  610000
3  3600  680000
4  4000  725000

Summary Statistics:
               area          price
count  1005.000000    1005.000000
mean   2728.221891  328780.261692
std    1307.687692  159758.985887
min     500.000000   33346.000000
25%    1583.000000  192995.000000
50%    2699.000000  326359.000000
75%    3877.000000  467028.000000
max    4999.000000  725000.000000

Missing Values:
 area     0
price    0
dtype: int64
```

Multiple Linear Regression

```python
features = ["area", "bedrooms", "age"]
model = LinearRegressionModel("homeprices-m.csv", x_columns=features, y_column="price")
model.load_and_explore_data()
model.prepare_data()
model.train_model()
model.evaluate_model()
model.visualize_results()

```

[3] ✓ 0.4s Python

```
First 5 Rows:
    area  bedrooms  age   price
0  1460       3.0    8  195536
1  4372       1.0   33  526139
2  3692       NaN    6  524207
3  1066       3.0   39  111925
4  4044       2.0    4  511168

Summary Statistics:
               area    bedrooms          age          price
count  1000.000000  980.000000  1000.000000    1000.000000
mean   2854.807000    3.462245    20.056000  356472.815000
std    1221.734084    1.699487    11.195723  149091.638174
min     603.000000    1.000000     1.000000   36587.000000
25%    1816.250000    2.000000    10.000000  238Save As00
50%    2871.500000    3.000000    21.000000  364508.500000
75%    3881.500000    5.000000    30.000000  477931.500000
max    4989.000000    6.000000    39.000000  694382.000000

Missing Values:
 area         0
bedrooms    20
age          0
price        0
dtype: int64
```

2.2 Data Preparation

Data preparation steps were performed for both datasets:

Simple Linear Regression Dataset (homeprices.csv)

- The dataset included two columns: area and price.
- No missing values were found.
- The feature (X = area) and target (y = price) were separated.
- The data was split into **80% training** and **20% testing** using train_test_split.

```python
    def prepare_data(self, test_size: float = 0.2, random_state: int = 42) -> None:
        """Handle missing values (drop rows) and split into train/test sets."""
        if self.df.isnull().values.any():
            print("⚠ Missing values detected — dropping rows with nulls.")
            self.df.dropna(inplace=True)
            print(f"Missing values dropped. New shape: {self.df.shape}")
        else:
            print("No missing values found.")

        X = self.df[self.x_columns].values
        y = self.df[self.y_column].values
        self.X_train, self.X_test, self.y_train, self.y_test = train_test_split(
            X, y, test_size=test_size, random_state=random_state
        )

```

```
No missing values found.
MAE: 15666.65
MSE: 380758008.43
RMSE: 19513.02
R² Score: 0.99
```

Multiple Linear Regression Dataset (homeprices-m.csv)

- This dataset included area, bedrooms, age, and price.
- A small percentage of rows contained missing values in the bedrooms column.
    - These rows were removed to simplify preprocessing.
- Features (X = area, bedrooms, age) and target (y = price) were defined.
- The cleaned data was then split into **80% training** and **20% testing**.

```python
def prepare_data(self, test_size: float = 0.2, random_state: int = 42) -> None:
    """Handle missing values (drop rows) and split into train/test sets."""
    if self.df.isnull().values.any():
        print("⚠ Missing values detected — dropping rows with nulls.")
        self.df.dropna(inplace=True)
        print(f"Missing values dropped. New shape: {self.df.shape}")
    else:
        print("No missing values found.")

    X = self.df[self.x_columns].values
    y = self.df[self.y_column].values
    self.X_train, self.X_test, self.y_train, self.y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state
    )

```

```
⚠ Missing values detected — dropping rows with nulls.
Missing values dropped. New shape: (980, 4)
MAE: 25027.55
MSE: 997798405.43
RMSE: 31587.95
R² Score: 0.96
```

In the multiple regression dataset, rows containing missing values in the bedrooms column were dropped to ensure clean input for model training.

After dropping missing values, the dataset contained 980 rows. Handling missing data before modeling ensures model accuracy and avoids training errors.

2.3 Model Development

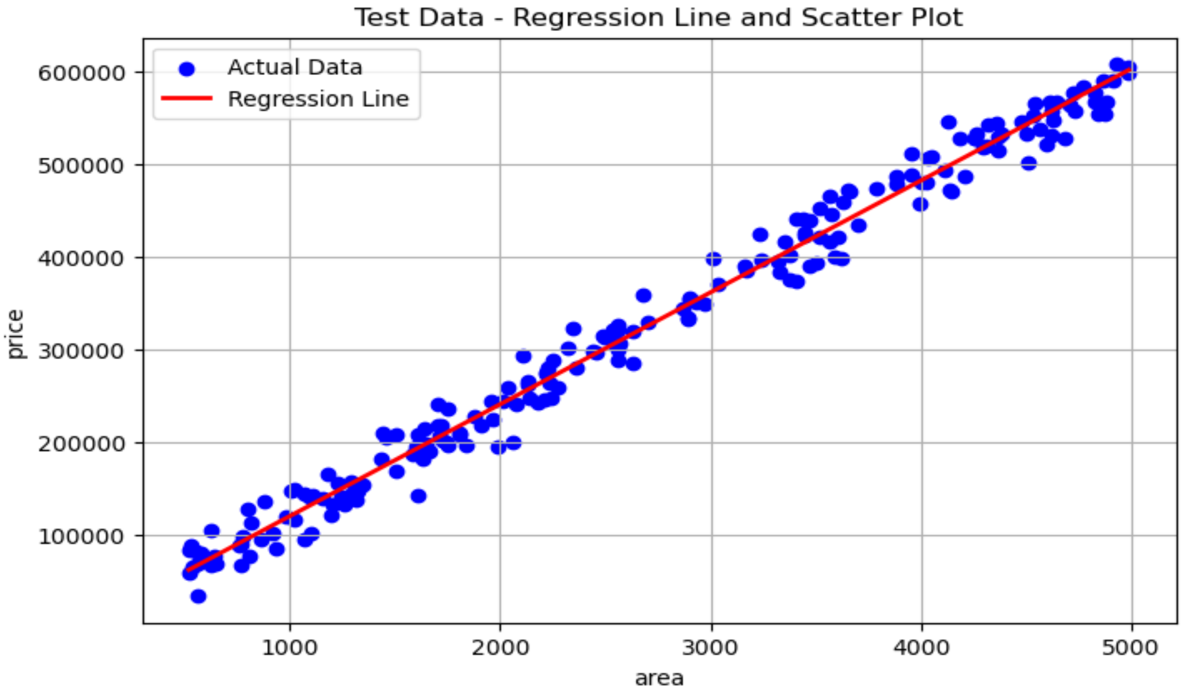
Two regression models were developed using Scikit-learn's LinearRegression:

a. Simple Linear Regression

- **Input**: area
- **Target**: price

The model was trained using the training set and tested on unseen data.

```python
def train_model(self) -> None:
    """Train the linear regression model."""
    self.model.fit(self.X_train, self.y_train)
```

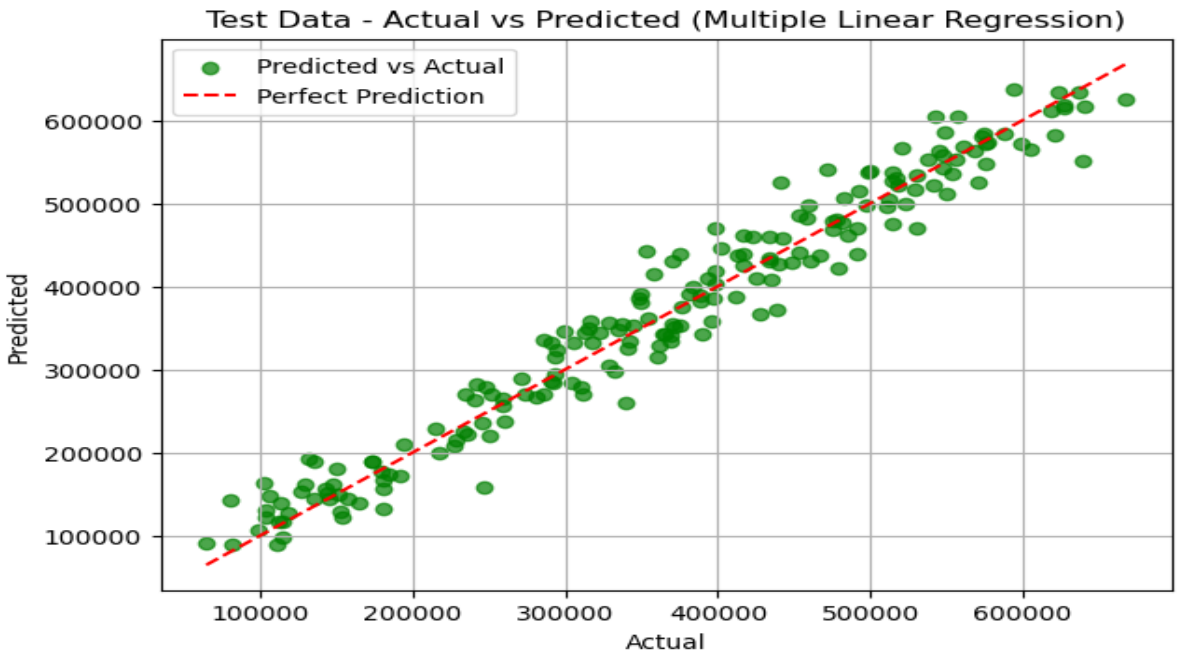


b. Multiple Linear Regression

- **Inputs**: area, bedrooms, age
- **Target**: price

This model aimed to improve prediction accuracy by using multiple input variables.

```
def train_model(self) -> None:
    """Train the linear regression model."""
    self.model.fit(self.X_train, self.y_train)
```



2.4 Model Evaluation

Each model was evaluated using:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score (explained variance)

Evaluation Results - Simple Linear Regression

```
MAE: 15666.65
MSE: 380758008.43
RMSE: 19513.02
R² Score: 0.99
```

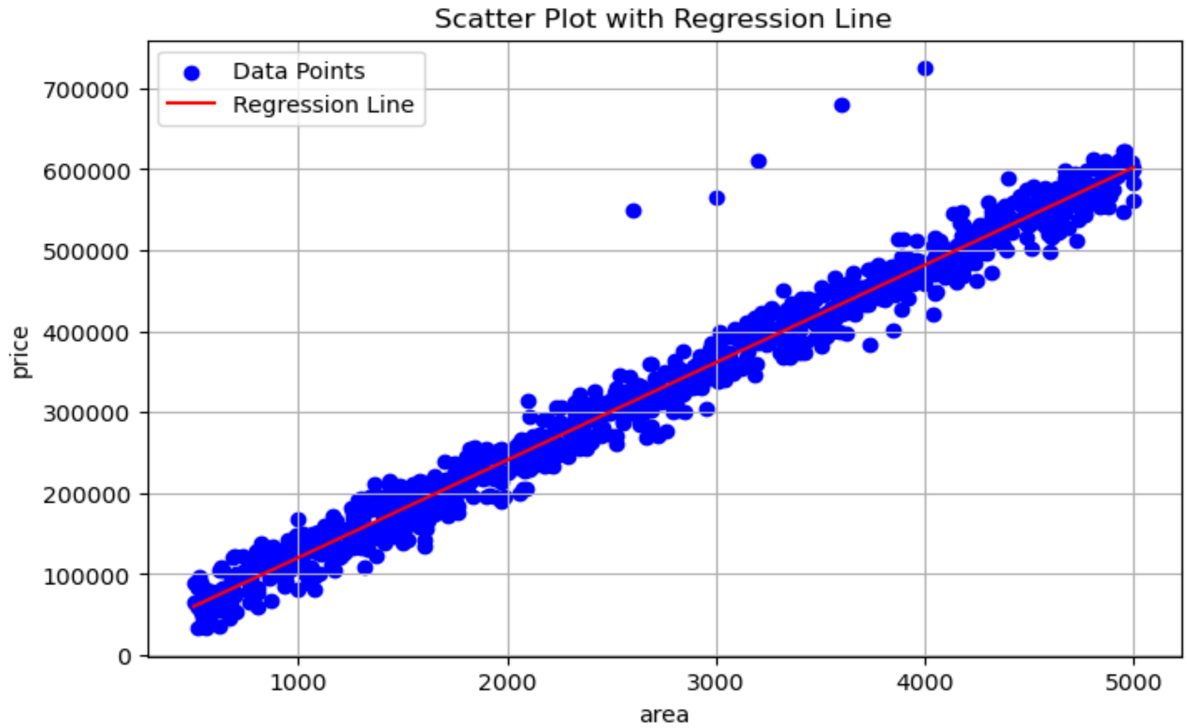
Evaluation Results - Multiple Linear Regression

```
MAE: 25027.55
MSE: 997798405.43
RMSE: 31587.95
R² Score: 0.96
```

2.5 Visualizations

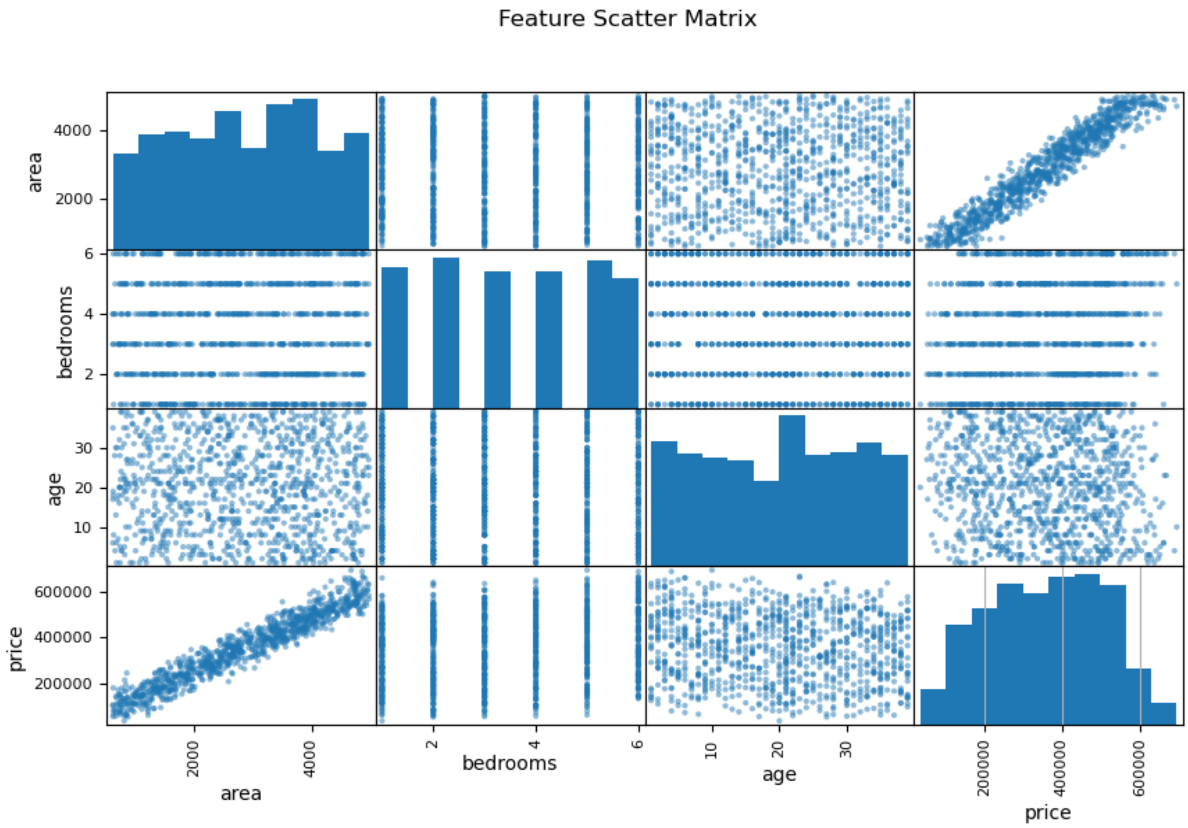
a. **Simple Linear Regression Plot**

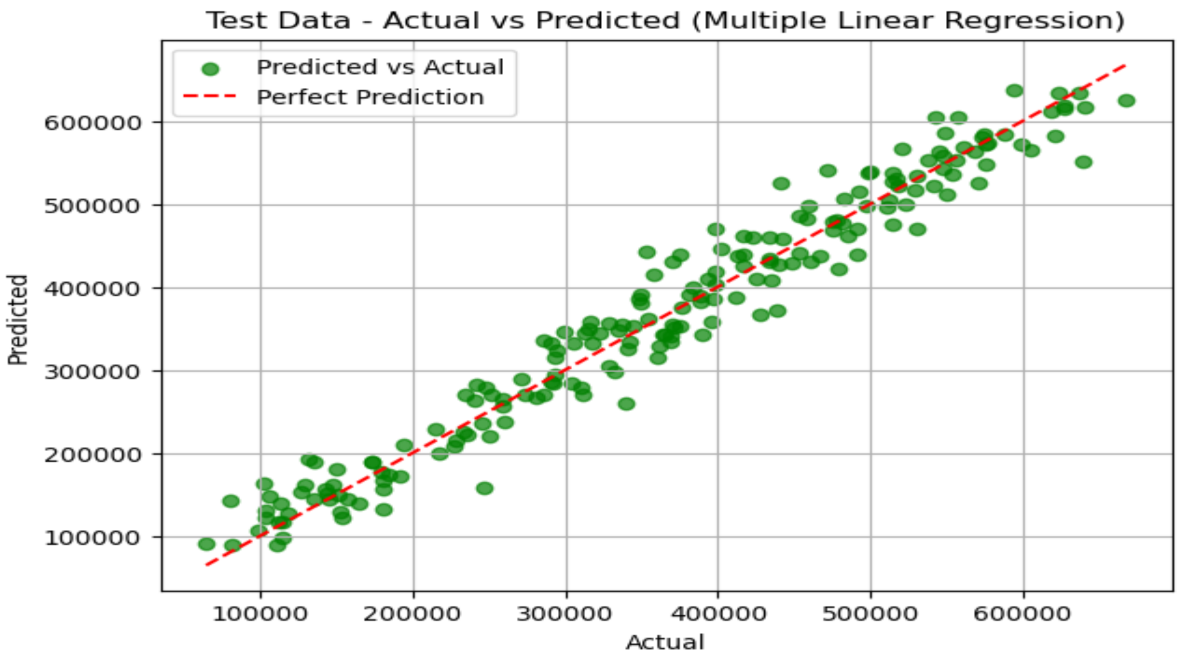
   A scatter plot of area vs. price was generated, with the regression line overlaid to show model fit.



b. Multiple Linear Regression Plot

An actual vs. predicted plot was created to visualize model performance. The diagonal reference line indicates perfect prediction.





2.6 Publishing and Sharing

The final notebook was uploaded to Google Drive (VS Code Notebook) and made publicly accessible. All code, datasets, results, and visualizations can be accessed for review.

🔗 VS Code Notebook Link:

[https://drive.google.com/file/d/1D5hpWqOiPZxpW4_7gyYXQjgYrU38ibui/view?usp=sharing]

3.0 PORTFOLIO WEBSITE

Website Link: *[https://amidu-dabor.github.io/]*

4.0 CONCLUSION

This project provided hands-on experience in building and evaluating linear regression models in Python. By working with real-world housing data, I gained insight into how model complexity and feature inclusion affect predictive performance. I also practiced proper model evaluation and visualization techniques. The work has been documented, published, and added to my data analytics portfolio to demonstrate both technical and analytical skills.

5.0 REFERENCES

1. Scikit-learn Documentation: https://scikit-learn.org
2. Pandas Documentation: https://pandas.pydata.org
3. Matplotlib Documentation: https://matplotlib.org
4. Cyber Shujaa LMS Materials